

Actividad 1: Introducción a Prolog

FORMA DE ENTREGA

- Rellenar el documento de entrega y guardar como **PDF**.
- Los ejercicios deben estar correctamente numerados.
- Se debe mantener un estilo (tamaño, tipo de letra, maquetación) coherente y elegante. Texto justificado, imágenes centradas.

Antes de comenzar...

Para la realización de la actividad tendremos que instalar el entorno de programación con el que ejecutaremos los programas, [SWI-Prolog](#).

1. Introducción a la programación lógica

Cuando hablamos de paradigmas de programación, los más conocidos y habituales de encontrar son la programación imperativa y la orientada a objetos, siendo C el mayor exponente del primero y Java del segundo. Sin embargo, existen otros paradigmas de programación totalmente diferentes a ellos, entre los que se encuentran los lenguajes lógicos y declarativos.

En la programación lógica en vez de establecer un proceso de cómo se deben hacer las cosas para conseguir un resultado, se establecen una serie de hechos ciertos y reglas de inferencia y se deja que el programa decida cuál es la mejor manera de llegar al resultado deseado.

Por lo tanto, podemos decir que un **lenguaje de programación lógico** es un sistema de notación para escribir enunciados lógicos junto con algoritmos especificados para implementar las reglas de inferencias.

Los siguientes enunciados pueden considerarse lógicos:

Un caballo es un mamífero.

Un ser humano es un mamífero.

Los mamíferos tienen cuatro patas y ningún brazo, o dos patas y dos brazos.

Un caballo no tiene brazos.

Un ser humano tiene brazos.

Un ser humano no tiene patas.

En cálculo de predicados de primer orden, una posible interpretación de los enunciados anteriores podría escribirse de la siguiente manera:

mamífero(caballo).

mamífero(humano).

para toda x, mamífero(x) $\rightarrow$ (patas(x,4) y brazos(x,0)) o (patas(x,2) y brazos(x,2)).

brazos(caballo,0).

no brazos(humano,0).

patas(humano,0).

[BDIA- MIAR] UT3 Sistemas basados en el conocimiento

Aplicando las reglas de inferencia del cálculo de predicados de primer orden podríamos considerar que los cinco primeros enunciados son axiomas (o sea, verdaderos). Y también permiten demostrar que el último enunciado es falso.

Efectivamente, sabiendo que una regla de inferencia típica de la lógica es que de los enunciados $a \rightarrow b$ y $b \rightarrow c$ podemos derivar el enunciado $a \rightarrow c$, es posible seguir la línea de pensamiento siguiente:

patas(caballo,4).
brazos(caballo,0).
patas(humano,2).
brazos(humano,2).

En lenguaje matemático, si tomamos los cinco primeros enunciados anteriores como axiomas (recordemos, como verdaderos), entonces estos cuatro últimos enunciados pasan a ser considerados **teoremas**. La comprobación de estos enunciados a partir de los axiomas puede considerarse como el cálculo del número de brazos y patas de un caballo o de un humano. O lo que es lo mismo, el conjunto de enunciados anteriores puede considerarse como la representación del cómputo potencial de todas las consecuencias lógicas de dichos enunciados.

Esta es la esencia última de la programación lógica: una colección de enunciados que se supone son ciertos y a partir de los cuales se deriva un hecho deseado mediante la aplicación de reglas de inferencia de alguna manera automatizada.

El conjunto de enunciados lógicos que se toman como axiomas pueden considerarse como el programa lógico y el enunciado que va a derivarse sería la ‘entrada’ que inicia el cómputo del programa. A estas últimas entradas es a lo que se suele llamar *consulta* o *meta*.

Dado el conjunto de axiomas anteriores, para saber cuántas patas tiene un ser humano haríamos una consulta de la siguiente manera:

¿Existe una y tal que y es el número de patas de un ser humano?

O en cálculo de predicados:

¿Existe y , patas(humano, y)?.

Se llaman cláusulas Horn a los enunciados expresados de la forma:

$a_1 \text{ y } a_2 \text{ y } a_3 \dots \text{ y } a_n \rightarrow b$

Donde cada a_n es un enunciado simple (no usa cuantificadores o conectores ‘o’). Por lo tanto, de la anterior expresión se deriva que b sólo será cierto si todos los a_n son ciertos.

Una expresión de la siguiente forma:

$\rightarrow b$

También es una cláusula Horn e implica que b siempre es verdadero, o lo que es lo mismo, que b es un axioma. Normalmente se escribe sin el $\rightarrow$.

La mayoría de los enunciados lógicos pueden expresarse mediante cláusulas Horn (aunque no todos ellos). La idea básica cuando escribamos las cláusulas será, por tanto, eliminar los conectores ‘o’ y suplir la carencia de cuantificadores suponiendo que las variables que aparecen en la cabeza de una cláusula están universalmente cuantificadas mientras que las que aparecen en el cuerpo (pero no en la cabeza) están existencialmente cuantificadas.

Veamos un ejemplo. Los siguientes enunciados están escritos en cálculo de predicados de primer orden:

[BDIA- MIAR] UT3 Sistemas basados en el conocimiento

natural(0).

para toda x , natural(x) $\rightarrow$ natural(sucesor(x)).

Establecen primero que 0 es un número natural y posteriormente que todos los sucesores de un número natural también son números naturales. Como hemos establecido como axioma que 0 es un número natural, todos los números posteriores también lo serán, pero no así los números enteros negativos.

Estos enunciados podemos convertirlos en cláusulas de Horn desprendiéndonos de los cuantificadores:

natural(0).

natural(x) $\rightarrow$ natural(sucesor(x)).

Veamos otro ejemplo, en este caso el algoritmo para calcular el máximo común divisor de dos números enteros positivos u y v :

El mcd de u y 0 es u .

El mcd de u y de v , si v no es 0, es el mismo que el mcd de v y el resto de u entre v .

Lo anterior en cálculo de predicados de primer orden puede expresarse:

para toda u , mcd($u, 0, u$).

para toda u , para todas v , para todas w , no_cero(v) y mcd($v, u \bmod v, w$) $\rightarrow$ mcd(u, v, w).

En este caso mcd(u, v, w) es un predicado que dice que w es el mcd de u y v .

Traducido a cláusulas Horn, eliminando los cuantificadores:

mcd($u, 0, u$).

no_cero(v) y mcd($v, u \bmod v, w$) $\rightarrow$ mcd(u, v, w).

En los ejemplos anteriores manejamos variables universalmente cuantificadas (para toda x , para toda u). Veamos ahora un ejemplo con variable existencialmente cuantificada en el cuerpo:

x es un abuelo de y si x es el padre de alguien que es padre de y .

En cálculo de predicados:

para toda x , para todas y , (existe z , padre(x, z) y padre(z, y)) $\rightarrow$ abuelo(x, y).

Y como cláusula Horn:

padre(x, z) y padre(z, y) $\rightarrow$ abuelo(x, y).

Por último, un ejemplo de cómo eliminar conectores 'o':

para toda x , si x es un mamífero, entonces x tiene dos o cuatro patas.

Traducido a cálculo de predicados:

para todas x , mamífero(x) $\rightarrow$ patas($x, 2$) o patas($x, 4$).

Podemos expresar lo anterior con cláusulas Horn como:

mamífero(x) y no_patas($x, 2$) $\rightarrow$ patas($x, 4$).

mamífero(x) y no_patas($x, 4$) $\rightarrow$ patas($x, 2$).

En este caso, cuantos más conectores aparezcan a la derecha de un conector ' $\rightarrow$ ' más difícil será traducirlo a cláusulas Horn.

Las cláusulas Horn son especialmente interesantes para los sistemas automáticos de deducción (como la programación lógica) porque puede dárseles una **interpretación procedimental**. Si escribimos una cláusula Horn en orden inverso

$b \leftarrow a_1 \text{ y } a_2 \text{ y } a_3 \dots \text{ y } a_n$

[BDIA- MIAR] UT3 Sistemas basados en el conocimiento

obtenemos prácticamente la definición de un procedimiento. A partir de ahora las escribiremos de este modo.

La mayoría de los sistemas de programación lógica escriben cláusulas Horn ‘al revés’ y prescinden del conector ‘y’ para sustituirlo por una ‘,’. Las cláusulas del mcd anteriores se verían de la siguiente forma:

$$\text{mcd}(u, 0, u).$$

$$\text{mcd}(u, v, w) \leftarrow \text{no_cero}(v), \text{mcd}(v, u \bmod v, w).$$

Esto ya se va pareciendo sospechosamente a una expresión de lenguaje de programación al uso.

Para saber cómo podemos expresar las consultas (o metas) como cláusulas de Horn, tenemos que introducir un último concepto más, la regla de resolución, que es el principio que aplican los sistemas de programación lógica para derivar nuevos enunciados a partir de otros enunciados.

La **regla de resolución** dice que si tenemos dos cláusulas Horn y podemos emparejar la cabeza de la primera con uno de los enunciados en el cuerpo de la segunda, entonces puede utilizarse la primera cláusula para reemplazar su cabeza en la segunda cláusula utilizando su cuerpo:

$$a \leftarrow a_1, \dots, a_n$$

$$b \leftarrow b_1, \dots, b_n$$

donde a es igual a algún b_n podemos inferir la cláusula

$$b \leftarrow b_1, \dots, b_{n-1}, a_1, \dots, a_n, b_{n+1}, \dots, b_n$$

Un ejemplo más sencillo (y ampliamente utilizado) es cuando sólo existen enunciados individuales en el cuerpo:

$$b \leftarrow a$$

$$c \leftarrow b$$

Entonces

$$c \leftarrow a$$

Si os fijáis, esta regla ya la hemos utilizado en el primer ejemplo que hemos visto.

Otra forma de expresar la regla sería de la siguiente manera:

$$b \leftarrow a$$

$$c \leftarrow b$$

$$b, c \leftarrow a, b$$

$$b, c \leftarrow a, b$$

$$c \leftarrow a$$

Ahora ya podemos vislumbrar la forma en que un sistema de programación lógica puede tratar una meta o una lista de metas.

Las consultas o metas son de hecho lo contrario a un axioma (que es un encabezado sin cuerpo, $\text{mamífero}(\text{humano}) \leftarrow$). Se expresan como una cláusula Horn sin cabeza ($\leftarrow \text{mamífero}(\text{humano})$).

El sistema de programación lógica intentará emparejar una meta (en el cuerpo de una cláusula sin cabeza) con la cabeza de una cláusula conocida, de manera que podremos sustituir la meta por el cuerpo de la cláusula con la que la hemos emparejado:

$$\leftarrow a$$

$$a \leftarrow a_1, \dots, a_n$$

$$\leftarrow a_1, \dots, a_n$$

[BDIA- MIAR] UT3 Sistemas basados en el conocimiento

De esta manera se irán creando nuevas metas llamadas submetas con las cuales se intentará usar también la regla de resolución. Si el sistema consigue eliminar todas las metas derivando en una cláusula Horn vacía se habrá comprobado la validez del enunciado original.

Veamos un ejemplo simple. Dadas las reglas:

```
patas(x,2) <- mamífero(x), brazos(x,2).
patas(x,4) <- mamífero(x), brazos(x,0).
mamífero(caballo).
brazos(caballo,0).
```

Si suministramos la consulta

```
<- patas(caballo,4).
```

Podemos aplicar la regla de resolución de la segunda forma que hemos visto antes:

```
patas(x,4) <- mamífero(x), brazos(x,0).
<- patas(caballo,4).
```

```
patas(x,4) <- mamífero(x), brazos(x,0), patas(caballo,4).
```

Ahora emparejamos la variable *x* con *caballo* de manera que en la anterior expresión sustituimos todas las *x* por *caballo*.

```
patas(caballo,4) <- mamífero(caballo), brazos(caballo,0), patas(caballo,4).
patas(caballo,4) <- mamífero(caballo), brazos(caballo,0), patas(caballo,4).
<- mamífero(caballo), brazos(caballo,0).
```

Y tanto *mamíferos(caballo)* como *brazos(caballo,0)* son axiomas con lo cual nuestra consulta queda cancelada.

Más exactamente, si aplicamos la misma regla que antes con ambos (recuerda que pueden escribirse como *mamíferos(caballo) <-* y *brazos(caballo,0) <-*) obtenemos la cancelación.

```
mamífero(caballo) <-
<- mamífero(caballo), brazos(caballo,0).
mamífero(caballo) <- mamífero(caballo), brazos(caballo,0).
mamífero(caballo) <- mamífero(caballo), brazos(caballo,0).

<- brazos(caballo,0).
brazos(caballo,0) <-
brazos(caballo,0) <- brazos(caballo,0).
brazos(caballo,0) <- brazos(caballo,0).
```

Como hemos llegado a un enunciado vacío, nuestra consulta original es verdadera.

2. El lenguaje Prolog

El lenguaje Prolog utiliza una notación casi idéntica a lo que acabamos de ver para su programación. Tan sólo reemplaza las flechas (<-) por dos puntos seguidos de un guion (: -). Las variables empiezan siempre por mayúscula o con un guion bajo ('_x' o 'X'). Las cláusulas finalizan con un punto '.' El conector 'o' se sustituye por ';' (aunque apenas se usa fuera de resultados de consultas ya que no debe formar parte de cláusulas Horn) y el conector 'y' se sustituye por ','

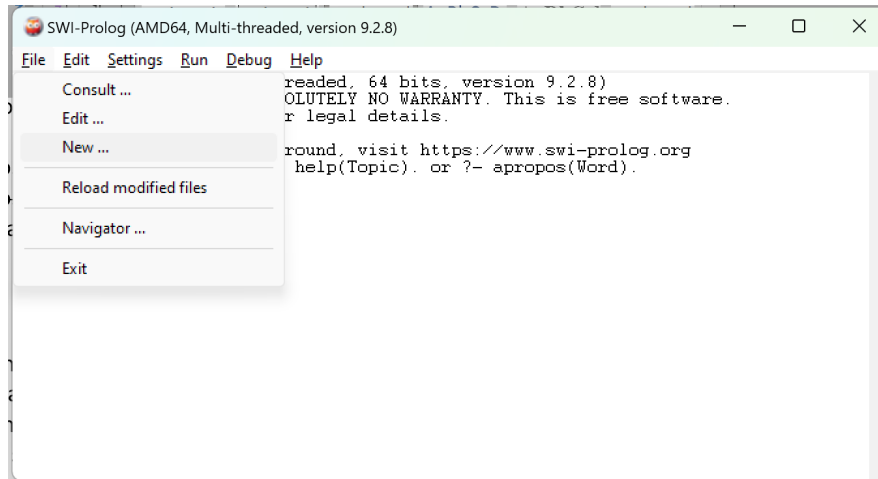
```
ancestro(X,Y) :- padre(X,Z), ancestro(Z,Y).
ancestro(X,X).
padre(amaia, jon).
```

[BDIA- MIAR] UT3 Sistemas basados en el conocimiento

Por lo general introduciremos en una base de conocimiento las cláusulas y realizaremos las consultas sobre esa base de conocimiento.

Veamos un ejemplo sencillo utilizando el intérprete SWI-Prolog. Te recomiendo que vayas haciéndolo a la vez que lo lees.

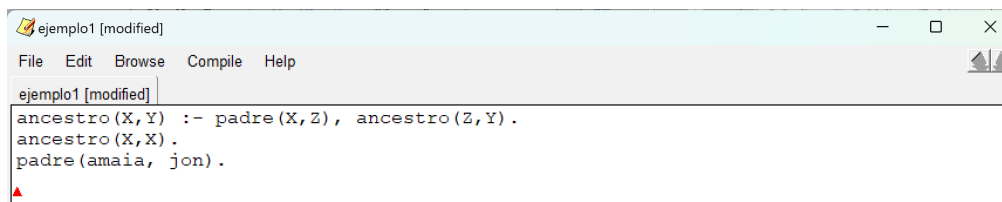
Primero vamos a crear una base de conocimiento en la que introduciremos nuestras cláusulas. Para ello podemos ir a *File -> New...* y crear el nuevo archivo



`?- edit(file(ejemplo1)).`

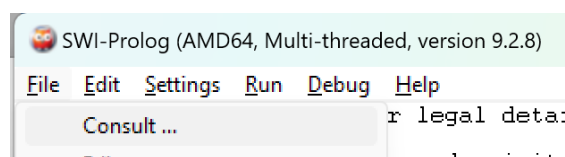
o escribir directamente en la ventana del intérprete en donde ejemplo1 es el nombre de nuestro nuevo archivo. Recuerda poner el punto al final de cada cláusula.

En ambos casos nos abrirá otra ventana en la que podremos introducir nuestras cláusulas. Cuando hayamos acabado tendremos que guardar el archivo si queremos utilizarlo para realizar consultas (*File -> Save Buffer* en la ventana de la base de conocimiento o las teclas *ctrl+s*).



Vamos a escribir las cláusulas que hemos visto un poco más arriba y guardamos.

Seguidamente vamos a realizar consultas sobre ellas. Tendremos que indicar al programa que queremos operar sobre esa base de conocimiento. Para ello en la ventana del intérprete podemos ir a *File -> Consult...* y elegir el archivo que acabamos de generar



```
?- consult(ejemplo1).
```

O escribir en el intérprete `consult(ejemplo1).` en donde `ejemplo1` es el nombre que le he dado al archivo de la base de conocimiento. En este caso veremos que nos devuelve por pantalla **true** si ha cargado el archivo. En caso contrario devolverá un mensaje de error.

```
?- consult(ejemplo1).  
true.  
?- ■
```

Vamos a realizar nuestra primera consulta. Vamos a preguntar si Amaia es un ancestro de Jon.

```
?- ancestro(amaia,jon).  
true
```

Que nos devuelve **true**. Por el contrario, si hacemos la consulta al revés

```
?- ancestro(jon,amaia).  
false.
```

Intuitivamente vemos que si Amaia es ‘padre’ para Jon, también será ancestro suyo, sin embargo lo contrario no será cierto. Puedes intentar seguir el razonamiento mediante cláusulas de Horn de manera similar a como lo hemos hecho anteriormente, si quieres.

Vamos ahora a requerir a nuestro intérprete que nos diga cuáles son los ancestros de Jon.

```
?- ancestro(X,jon)  
X = amaia ■
```

Vemos que el cursor se queda esperando al final de la línea. Eso nos indica que hay más de un resultado. Si queremos ver el resto tendremos que escribir el signo ‘;’ que recordemos que es la conjunción ‘o’. Cuando ya no haya más resultados el intérprete quedará de nuevo preparado para recibir consultas.

```
?- ancestro(X,jon).  
X = amaia ;  
X = jon.  
?- ■
```

Pongamos un ejemplo un poco más extenso.

Vamos a crear una base de conocimiento con el siguiente contenido:

```
% nombres de personas  
persona(alfonso).  
persona(ana).  
persona(carlos).  
persona(carmen).  
persona(fernando).  
persona(isabel).  
persona(luis).  
persona(maria).  
% deportes y su clasificacion  
tipo_deporte(marcha,atletismo).  
tipo_deporte(vallas,atletismo).  
tipo_deporte(cross,atletismo).  
tipo_deporte(disco,atletismo).  
tipo_deporte(martillo,atletismo).  
tipo_deporte(salto,nieve).  
tipo_deporte(esqui_alpino,nieve).  
tipo_deporte(esqui_fondo,nieve).  
tipo_deporte(snowboard,nieve).
```

```
tipo_deporte(trineo,nieve).
tipo_deporte(futbol,equipo).
tipo_deporte(baloncesto,equipo).
tipo_deporte(balonmano,equipo).
tipo_deporte(voleibol,equipo).
tipo_deporte(waterpolo,equipo).
% deporte que practica cada uno y nivel
 practica_deporte(alfonso,categoria(baloncesto,nba)).
 practica_deporte(ana,categoria(vallas,regional)).
 practica_deporte(carlos,categoria(marcha,nacional)).
 practica_deporte(carmen,categoria(futbol,tercera)).
 practica_deporte(fernando,categoria(baloncesto,aficionado)).
 practica_deporte(isabel,categoria(cross,regional)).
 practica_deporte(luis,categoria(snowboard,aficionado)).
 practica_deporte(maria,categoria(waterpolo,aficionado)).
```

El símbolo ‘%’ es para los comentarios sobre las cláusulas y toda la línea posterior no es tenida en cuenta por el intérprete.

Carga el archivo para consultas y realiza las consultas que respondan a las siguientes preguntas:

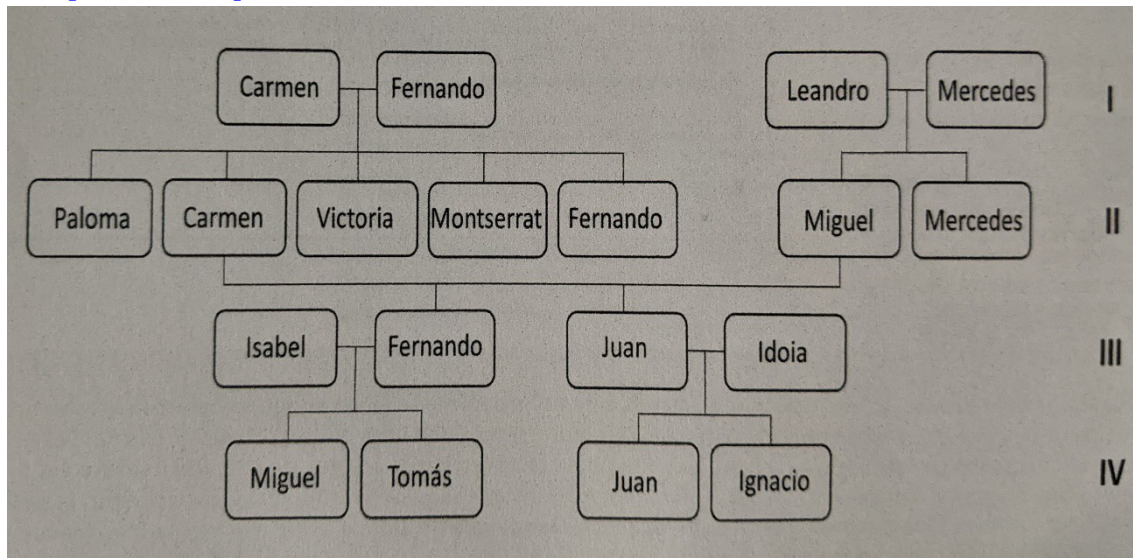
- ¿Qué personas hay definidas en nuestra base de conocimiento?
- ¿Qué deportes de nieve se han definido?
- ¿Alguien practica snowboard y a qué nivel?
- ¿Alguien practica baloncesto a nivel de NBA?

Al final del documento encontrarás las soluciones. Intenta hacerlas por tu cuenta primero.

3. Árbol genealógico

Vamos a representar en una base de conocimiento el árbol genealógico de una familia. Es posible representar en Prolog los integrantes de la familia y sus relaciones, de manera que luego el intérprete será capaz de responder preguntas relativas a las relaciones familiares.

En el siguiente diagrama tenemos el árbol familiar representado de manera gráfica. Podemos apreciar cómo no tenemos registrados los apellidos y hay nombres que se repiten. Lo que vamos a hacer para distinguir entre ellos es completar el nombre de cada uno de ellos con la generación a la que pertenecen, que se muestra a la derecha del gráfico.



En primer lugar tendréis que añadir todos los **integrantes** de la familia a la base de conocimiento. Lo haremos indicando su **sexo**.

```

arbol.pl [modified]
File Edit Browse Compile Prolog
arbol.pl [modified]
% nivel 1
hombre(fernandoI).
mujer(carmenI).

```

Una vez incluidos todos los integrantes de la familia tendremos que incluir todas las posibles relaciones entre ellos. Por ejemplo, indicaremos los **progenitores** de cada uno de los integrantes.

```

% relaciones de progenitura
progenitor(fernandoI, fernandoII).

```

También tendremos que registrar los **matrimonios** entre los miembros de la familia.

Por último, tendremos que incluir las reglas que rigen las relaciones de parentesco. La primera puede ser la relación de parentesco **padre de**. Podemos decir que un padre es aquel que es progenitor de una persona y de sexo masculino.

```

% relaciones de parentesco
padre(Padre, Hijo) :- hombre(Padre), progenitor(Padre, Hijo).

```

De manera equivalente añadiremos las relaciones **madre de**. Se considera que dos personas son **hermanos** si comparten progenitores y no son la misma persona. Si la persona que cumple lo anterior es de sexo masculino se considera que es **hermano**, y si

[BDIA- MIAR] UT3 Sistemas basados en el conocimiento

es de sexo femenino será **hermana**. Tendremos que generar también las relaciones de **esposo**, **esposa**, **nieto** y **nieta**. Genera al menos las relaciones que están en negrita.

Comprueba que nuestra base de conocimiento es capaz de responder de manera correcta a todas las preguntas referentes a las relaciones de parentesco que hemos creado.

Por ejemplo, `nieto(juanIII, Quien)` . debería ser capaz de darnos el listado de todos los nitos de `juanIII`. O `hermanos(carmenII, fernandoII)` . Debería responder con **true**.

Rellena el documento de entrega respondiendo a las preguntas e incluyendo las capturas de pantalla solicitadas en el.

Soluciones a las cuestiones finales del apartado 2:

```
?- persona(Alguien).
Alguien = alfonso ;
Alguien = ana ;
Alguien = carlos ;
Alguien = carmen ;
Alguien = fernando ;
Alguien = isabel ;
Alguien = luis ;
Alguien = maria.
```

?-

```
?- tipo_deporte(Nombres, nieve)
Nombres = salto ;
Nombres = esqui_alpino ;
Nombres = esqui_fondo ;
Nombres = snowboard ;
Nombres = trineo.
```

?-

```
?- practica_deporte(Alguien, categoria(snowboard, Nivel)).
Alguien = luis.
Nivel = aficionado.
```

?- ■

[BDIA- MIAR] UT3 Sistemas basados en el conocimiento

```
?- practica_deporte(Alguien,categoria(baloncesto,nba)).  
Alguien = alfonso ;  
false.
```

?- ■
